Project Proposal: Does LightGCN Favor Popular Items?

Team: Group 5 Members: Sachi Vyas, Sakhi Patel, Vaishvi Patel
CSC 591/791-038: Machine Learning for Graphs

1 Problem

The LightGCN paper describes recommender systems as widely used to filter information for each user [1]. If a recommender keeps suggesting the same popular items, other items, such as products from small sellers or songs from independent artists, get little attention.

LightGCN already accounts for item popularity in part. Its normalization scales each neighbor's contribution down by the square root of that neighbor's number of connections (Eq. 3), and Section 3.2.3 notes that sharing a less popular item creates stronger smoothing between two users than sharing a popular one. However, popular items still connect to many users, so they appear in many aggregation steps. The paper reports accuracy, training loss, and embedding smoothness (Table 6), but not which items end up recommended, so the net effect is unknown. Our first goal is to measure it.

Our question: How do LightGCN's number of layers, layer weights, and normalization affect how much it favors popular items, and how much accuracy is lost if recommendations are spread more evenly?

2 Background from the Three Papers

LightGCN [1]. Each layer updates a user's embedding by adding up the embeddings of the items they interacted with, each divided by the square root of both nodes' number of connections (Eq. 3). The authors use this normalization to keep embeddings from growing in scale across layers (Section 3.1.1), not to reduce popularity bias. The final embedding averages all layers with equal weights $\alpha_k = 1/(K+1)$ (Eq. 4):

$$e_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|}\sqrt{|\mathcal{N}_i|}} e_i^{(k)}, \qquad e_u = \sum_{k=0}^{K} \alpha_k \, e_u^{(k)}. \tag{1}$$

Items are updated the same way from their users. Three results shape our plan: the normalization choice matters (Table 5: Gowalla recall@20 ranges from 0.1830 down to 0.1420 across variants); without combining layers ("LightGCN-single"), results usually peak at 2 layers and then drop due to over-smoothing (Section 4.4.1); and the equal layer weights were not tuned (Section 4.3), with personalized weights listed as future work (Section 6).

SGC [2]. Repeatedly mixing neighbor information acts like a smoothing filter that keeps broad patterns and removes fine details (Section 3.2). Supplementary Lemma 2 shows that the "smoothest" pattern on a graph is proportional to the square root of each node's degree, and the proof of Lemma 3 links this to the matrix LightGCN uses. This suggests that with more layers, embeddings of high-degree (popular) items may grow longer, which could raise their scores. SGC's analysis uses self-loops, which LightGCN removes (Section 3.1.1), and LightGCN's layer combination keeps the layer-0 embedding, so we treat this only as an idea to test. Smoothing improves accuracy (LightGCN Section 4.4.3); we study only its side effect on exposure.

APPNP [3]. Mixing for too many steps makes every node's representation depend on the whole graph rather than on where it started (Section 2). APPNP keeps part of each node's original information at every step. LightGCN shows its layer weights can reproduce APPNP (Section 3.2.2) by setting $\alpha_k = \beta(1-\beta)^k$ for $k < K$ and $\alpha_K = (1-\beta)^K$, where a larger β keeps more of the original embedding. We also borrow APPNP's testing practice (Section 5): repeated runs and a validation set kept separate from the test set.

3 Research Questions and Hypotheses

We will report results whether or not they support these hypotheses.

1. **Layers:** Does LightGCN favor popular items more with more layers (1 to 4)? *Hypothesis:* Yes, more clearly for LightGCN-single; combining layers reduces this.
2. **Layer weights:** Do APPNP-style weights change which items get recommended? *Hypothesis:* Larger β leads to more less-popular items being recommended.
3. **Normalization:** If we reduce popular neighbors' influence *further* than LightGCN already does, do less popular items get recommended more, and at what accuracy cost? *Hypothesis:* Yes, with some drop in recall@20.

4. **Why:** As layers increase, do item embedding lengths (norms) track item degree more closely, and does this match the bias we measure?

4 Method

We use the PyTorch implementation linked in the LightGCN paper with its settings: BPR loss, embedding size 64, Adam with learning rate 0.001, batch size 1024 (Section 4.1.2). Unless stated, we use 3 layers, which the paper found satisfactory.

Step 1 (Q1, Q2). Train LightGCN and LightGCN-single with 1–4 layers. Then, at 3 layers, replace the equal weights with APPNP-style weights for $\beta \in \{0.1, 0.2, 0.5\}$.

Step 2 (Q3). Split LightGCN's normalization into two exponents, a for the node's own connections and b for the neighbor's (applied the same way when items aggregate from users):

$$e_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{|\mathcal{N}_u|^a \, |\mathcal{N}_i|^b} \, e_i^{(k)}. \tag{2}$$

LightGCN uses $a = b = 0.5$, and all variants in its Table 5 fit this formula (e.g., the right-side L_1 variant is $a = 0$, $b = 1$). Table 5 already reports their accuracy, and the variants with $b = 1$ score lower than the default on all three datasets; it does not report which items they recommend. We test $b \in \{0.5, 0.75, 1.0\}$, choose a on the validation set, and measure both accuracy and exposure. Raising b above 0.5 makes LightGCN's existing down-weighting of popular neighbors stronger.

Step 3 (comparison). Compare every change against the original LightGCN and matrix factorization (the 0-layer case, Section 4.2).

Step 4 (Q4). SGC's degree-based pattern has a simple closed form (square root of degree), so no extra computation is needed. For each layer count, we measure the correlation between item embedding length (norm) and the square root of item degree, and between item degree and how often each item is recommended.

What is new. None of the three papers measure how evenly items are recommended; we measure how LightGCN's own design choices affect this.

5 Experimental Setup

Datasets. The same three datasets and train/test splits as the LightGCN paper (numbers from its Table 2):

Dataset	Users	Items	Interactions	How we use it
Gowalla	29,858	40,981	1,027,370	All experiments; first reproduce the 3-layer recall@20 of 0.1823 (Table 3).
Yelp2018	31,668	38,048	1,561,406	Selected experiments; LightGCN-single works best at 2 layers here.
Amazon-Book	52,643	91,599	2,984,108	Selected experiments; another normalization beat the default here (Table 5).

What we measure. Accuracy: recall@20 and NDCG@20, as in LightGCN. Bias: we call the top 20% of items by training interactions "popular" and report *tail share* (percent of top-20 recommendations that are less popular), *coverage* (percent of items recommended to at least one user), the *Gini coefficient* (how unevenly recommendations are spread; 0 = even), and *tail recall@20* (recall on test interactions with less popular items).

How we test. We hold out part of the training data as a validation set for tuning and early stopping, and use the test set only for final numbers. LightGCN needs about 1,000 epochs to converge (Section 4.1.2), so we use 3 random seeds per setup and report mean and standard deviation.

References

[1] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang. LightGCN: Simplifying and powering graph convolution network for recommendation. *SIGIR*, 2020.

[2] F. Wu, T. Zhang, A. H. Souza Jr., C. Fifty, T. Yu, and K. Q. Weinberger. Simplifying graph convolutional networks. *ICML*, 2019.

[3] J. Gasteiger, A. Bojchevski, and S. Günnemann. Predict then propagate: Graph neural networks meet personalized PageRank. *ICLR*, 2019.